B9 Experimental design

Generated from the same content files as the app. Each module lists its roadmap months. Solutions are in the appendix.

Contents

Track B: Python for data science 2
B9 Experimental design 3
B9-L1 Power and sample size, before fieldwork 3
B9-L2 Analyse an A/B test the way it was randomised 7
B9-L3 Peeking, novelty and other ways tests mislead 11
Track B best-practice checklist 17
Appendix: solutions and answer keys 18

Track B: Python for data science

Write Python and run analyses that are correct, reproducible and easy to review.

B9 Experimental design

Roadmap: Prerequisite for all months. Power, sample size and A/B test analysis.

B9-L1 Power and sample size, before fieldwork

Concept

A test with too few people cannot see the effect you care about. It returns "no significant difference", and the team concludes the change did nothing. Decide the sample size **before** fieldwork, from four inputs:

- **Baseline**: the current rate, for example 40% consideration.
- **Minimum detectable effect (MDE)**: the smallest change worth acting on, for example 5 points, to 45%.
- **Significance level** (alpha): usually 0.05, two-sided.
- **Power**: the chance of detecting the MDE if it is real. Usually 0.8.

For two proportions, the sample per group is:

```
z_a = NormalDist().inv_cdf(1 - alpha / 2)
z_b = NormalDist().inv_cdf(power)
p_bar = (p1 + p2) / 2
n = ((z_a * sqrt(2 * p_bar * (1 - p_bar)) + z_b * sqrt(p1 * (1 - p1) + p2
    * (1 - p2))) / (p2 - p1)) ** 2
```

Round up. For 40% to 45% at alpha 0.05 and power 0.8: **1,534 per group**. statsmodels 0.14 agrees: `NormalIndPower().solve_power(effect_size=proportion_effectsize(0.45, 0.40), alpha=0.05, power=0.8)` returns 1,533.05.

The MDE drives everything. Required `n` grows with the inverse square of the effect:

Change	n per group
40% to 45%	1,534
40% to 42%	9,493

Halving the effect you want to detect needs roughly four times the sample.

Two adjustments for survey work:

- **Design effect.** Weighted or clustered samples carry less information (lesson B8-L1). Multiply `n` by the expected design effect: with a design effect of 1.5, 1,533.05 × 1.5 rounds up to 2,300.
- **Subgroups.** If the decision needs the effect within each of four regions, each region needs the full `n`, not a quarter of it.

Write the power calculation into the research plan, with its inputs. If the budget cannot buy the `n`, say so before fieldwork, and agree a larger MDE or a different design.

Why it matters

A packaging test used 300 respondents per cell to compare purchase intent. The new pack scored 43% against 40%. The report said "no significant difference, keep the current pack". With 300 per cell, the test had about a 1 in 4 chance of detecting a real 5-point lift. The result said nothing about the pack. It said the test was too small.

Worked example: weak code

```
1 N_PER_CELL = 300  # what the budget allows
2 result = run_test(N_PER_CELL)
3 if result.p_value > 0.05:
2     conclusion = "No effect"
```

1. **Line 1.** Sample size from budget, with no check of what it can detect.
2. **Line 4.** Absence of evidence from an underpowered test read as evidence of absence.

Worked example: strong code

```
1  import math
2  from statistics import NormalDist
3
4
1  def n_per_group(p1: float, p2: float, alpha: float = 0.05, power: float =
     0.8) -> int:
6      """Sample per group to detect p1 vs p2 with a two-sided test."""
7      if p1 == p2:
2          raise ValueError("p1 and p2 must differ")
9      z_a = NormalDist().inv_cdf(1 - alpha / 2)
10     z_b = NormalDist().inv_cdf(power)
11     p_bar = (p1 + p2) / 2
12     top = z_a * math.sqrt(2 * p_bar * (1 - p_bar)) + z_b * math.sqrt(p1 *
     (1 - p1) + p2 * (1 - p2))
```

```
    return math.ceil((top / (p2 - p1)) ** 2)
```

1. **Line 5.** Every input to the decision is a named parameter.
2. **Line 8.** A zero effect needs infinite n. Fail clearly instead of dividing by zero.
3. **Line 13.** Round up: 1,533.05 means 1,534 people.

Fix baseline, MDE, alpha and power first, compute n, adjust for design effect and subgroups, then field.

Recap card

- **Four inputs to a sample size** Baseline, minimum detectable effect, alpha, power.
- **n per group, 40% vs 45%, alpha 0.05, power 0.8** 1,534.
- **Halve the effect you want to detect** About four times the sample.
- **Weighted or subgroup reads** Multiply n by the design effect. Each subgroup needs the full n.

Practice

Exercise 1. Multiple choice (level 1)

A test with 300 per cell finds no significant difference. Power for the effect you care about was 0.25. What can you conclude?

- a) There is no effect.
- b) Very little. The test was unlikely to detect the effect even if it were real.
- c) The effect is negative.
- d) The effect is exactly zero with 75% confidence.

Why?

- r1) Power 0.25 means a real effect of that size is missed 75% of the time.
- r2) Non-significant results are always meaningless.

Exercise 2. Predict the output (level 2)

Required n per group is 1,534 to detect 40% vs 45%. Roughly how many per group to detect 40% vs 42.5%, with the same alpha and power? The code prints the ratio of the two sample sizes, rounded.

```
import math
from statistics import NormalDist


def n_per_group(p1, p2, alpha=0.05, power=0.8):
    z_a = NormalDist().inv_cdf(1 - alpha / 2)
    z_b = NormalDist().inv_cdf(power)
```

```
    p_bar = (p1 + p2) / 2
    top = z_a * math.sqrt(2 * p_bar * (1 - p_bar)) + z_b * math.sqrt(p1 * (1 - p1) + p2 * (1 - p2))
    return math.ceil((top / (p2 - p1)) ** 2)


print(round(n_per_group(0.40, 0.425) / n_per_group(0.40, 0.45)))
```

Exercise 3. Fill the gap (level 2)

Compute the sample size with statsmodels for 40% vs 45%.

```
h = proportion_effectsize(0.45, 0.40)
n = NormalIndPower()._____(1)(effect_size=h, alpha=0.05, _____(2)=0.8)
```

Exercise 4. Write the function (level 3)

Write `n_per_group(p1, p2, alpha=0.05, power=0.8, design_effect=1.0) -> int`, using the formula in the concept with `statistics.NormalDist`. Multiply by `design_effect` before rounding up. Raise `ValueError` if `p1 == p2`, or if either share is outside (0, 1).

Your code must pass these checks:

- 40% vs 45% at alpha 0.05, power 0.8 must give 1,534, in either order.
- Use the alpha and power arguments in the z values.
- Multiply by design_effect before rounding up.
- Raise ValueError for equal shares or shares outside (0, 1).

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Sample per group to detect p1 vs p2 with a two-sided test.
def n_per_group(
    p1: float,
    p2: float,
    alpha: float = 0.05,
    power: float = 0.8,
    design_effect: float = 1.0,
) -> int: ...
```

Starter code:

```
import math


def n_per_group(p1, p2, alpha=0.05, power=0.8, design_effect=1.0):
    return 300
```

Exercise 5. Architecture design (level 4)

A client wants to know whether a new ad lifts consideration from 40% by at least 5 points, overall and within each of 4 regions. Weighting will give a design effect of about 1.5. The budget buys 4,000 completes. Choose the plan.

Options: 2 × 1,534 × 1.5 = 4,602: slightly above budget, 4,000 is plenty for any read.

1. Sample for the overall read
2. Regional reads
3. Recommendation

Sources

- NormalIndPower().solve_power(effect_size=proportion_effectsize(0.45, 0.40), alpha=0.05, power=0.8) returns 1533.05. (verified: Run on statsmodels 0.15.0 in this repository; matches the closed-form result of 1,534.)
- With 300 per cell, power to detect 40% vs 45% is 0.24. (verified: Computed with statsmodels NormalIndPower in this repository.)

B9-L2 Analyse an A/B test the way it was randomised

Concept

An A/B test compares groups that differ only by chance and by the change you made. Keep that property in the analysis.

1. Check the split first. If you assigned 50/50 and got 50,600 against 49,400, something went wrong: a redirect lost users, a bot filter hit one arm, a crash hit one version. A chi-square test on the counts detects this **sample ratio mismatch** (SRM). For those counts, p = 0.00015. A very small p-value here, below 0.001 by common practice, means stop and find the cause. The treatment effect is not trustworthy until you do.

```
from scipy.stats import chisquare
chisquare([50_600, 49_400]).pvalue        # 0.000148, SciPy 1.17
```

2. Analyse as randomised. Everyone assigned to B stays in B, even if they never saw the new page (intention to treat). Dropping "people who did not see it" from B, but not the matching people from A, breaks the randomisation.

3. Analyse at the unit of randomisation. If you randomised users, compute one outcome per user. Counting page views as independent inflates `n` and shrinks intervals.

4. Report the difference with its interval. For 1,150 of 10,000 in B against 1,000 of 10,000 in A, the lift is 1.5 points, with a 95% interval of 0.6 to 2.4 points. statsmodels 0.14:

```
from statsmodels.stats.proportion import confint_proportions_2indep
confint_proportions_2indep(1150, 10_000, 1000, 10_000)   # (0.0064, 0.0236)
```

5. Compare with the minimum effect you planned for (lesson B9-L1). An interval that lies wholly above the MDE is a clear win. One that straddles it needs more data or a judgement call, stated as such.

Fix the primary metric, the analysis and the stopping rule in writing before launch. Secondary metrics are exploratory and are reported as such (lesson B8-L3).

Why it matters

A checkout test showed a 4% lift in conversion. Nobody checked the split. Arm B had 3% fewer users than arm A, because a slow script in B timed out on older phones, and those users were logged as never arriving. They were also the users least likely to buy. The "lift" came from losing low-converting users from B. When the script was fixed, the lift disappeared.

An SRM check on day one would have flagged the missing 3%.

Worked example: weak code

```
b_seen = b[b["saw_new_page"]]
rate_a = a["converted"].mean()
rate_b = b_seen["converted"].mean()
print("lift", rate_b - rate_a)
```

1. **Line 1.** Filters one arm on a post-assignment event: no longer a fair comparison.
2. **Line 4.** No split check, no interval, no comparison with the planned effect.

Worked example: strong code

```
import math


def srm_p_value(n_a: int, n_b: int, expected_share_a: float = 0.5) -> float:
    """Chi-square p-value (1 df) for the observed split against the planned one."""
    n = n_a + n_b
```

```
    e_a, e_b = n * expected_share_a, n * (1 - expected_share_a)
    stat = (n_a - e_a) ** 2 / e_a + (n_b - e_b) ** 2 / e_b
    return math.erfc(math.sqrt(stat / 2))


def lift_interval(x_a: int, n_a: int, x_b: int, n_b: int, z: float =
  1.959964) -> tuple[float, float, float]:
    """Difference in conversion (B minus A) with a Wald interval, for
  large samples."""
    p_a, p_b = x_a / n_a, x_b / n_b
    se = math.sqrt(p_a * (1 - p_a) / n_a + p_b * (1 - p_b) / n_b)
    diff = p_b - p_a
    return diff, diff - z * se, diff + z * se
```

1. **Line 9.** Chi-square with 1 degree of freedom, in the standard library.
2. **Line 12.** Uses every assigned user, as randomised.

Check the split, keep everyone in their assigned arm, analyse per randomised unit, and report the lift with its interval.

Recap card

- **First step in any A/B analysis** Sample ratio mismatch check: chi-square on arm counts against the plan.
- **Intention to treat** Analyse everyone in the arm they were assigned to, whatever happened next.
- **Unit of analysis** The unit of randomisation, usually the user, not the page view.
- **Report a lift as** The difference with its interval, compared with the planned minimum effect.

Practice

Exercise 1. Multiple choice (level 1)

A 50/50 test ends with 50,600 users in A and 49,400 in B. The SRM check gives p = 0.00015. What next?

- a) Report the treatment effect. A 1% imbalance is small.
- b) Stop and find why B lost users before trusting any result.
- c) Reweight B up to 50,600 and continue.
- d) Drop 1,200 random users from A to balance.

Why?

- r1) A split that differs from the design means something removed users unevenly, and they may differ in outcome.
- r2) Any p-value below 0.05 invalidates a test.

Exercise 2. Predict the output (level 2)

What does this print?

```python
import math


def srm_p_value(n_a, n_b, expected_share_a=0.5):
    n = n_a + n_b
    e_a, e_b = n * expected_share_a, n * (1 - expected_share_a)
    stat = (n_a - e_a) ** 2 / e_a + (n_b - e_b) ** 2 / e_b
    return math.erfc(math.sqrt(stat / 2))


print(srm_p_value(5060, 4940) > 0.05, srm_p_value(50600, 49400) > 0.05)
```

Exercise 3. Put in order (level 2)

Order the analysis of a finished A/B test.

- Compare the interval with the planned minimum effect
- Check the split against the design
- Estimate the primary metric's lift with its interval
- Report secondary metrics as exploratory
- Aggregate to one outcome per randomised user

Exercise 4. Write the function (level 3)

Write `srm_p_value(n_a: int, n_b: int, expected_share_a: float = 0.5) -> float`. Compute the chi-square statistic with expected counts `n * expected_share_a` and `n * (1 - expected_share_a)`, and return the 1-degree-of-freedom p-value `math.erfc(math.sqrt(stat / 2))`. Raise `ValueError` if either count is negative, the total is 0, or `expected_share_a` is not strictly between 0 and 1.

Your code must pass these checks:

- The 50/50 p-values must match chi-square with 1 df: 0.000148 and 0.230.
- Use expected_share_a for the expected counts.
- Raise ValueError for a zero total, a negative count, or a share outside (0, 1).

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Sample ratio mismatch p-value: chi-square with 1 degree of freedom.
def srm_p_value(
    n_a: int,
    n_b: int,
    expected_share_a: float = 0.5,
```

```
) -> float: ...
```

Starter code:

```
def srm_p_value(n_a, n_b, expected_share_a=0.5):
    return abs(n_a - n_b) / (n_a + n_b)
```

Exercise 5. Spot the bug (level 4)

The analysis reports a larger lift than the product team expected. Tap the line that breaks the randomisation, and fix the function.

What you need to know.

`series.to_dict()` turns a pandas Series into a dict: the index labels become keys. After a `groupby(...).mean()`, the keys are the group names.

```
pd.Series([0.2, 0.4], index=["A", "B"]).to_dict()   # {'A': 0.2, 'B': 0.4}
```

```
import pandas as pd


def conversion_by_arm(users: pd.DataFrame) -> dict[str, float]:
    """Conversion rate per assigned arm. One row per randomised user."""
    analysed = users[(users["arm"] == "A") | users["saw_new_page"]]
    return analysed.groupby("arm")["converted"].mean().to_dict()
```

Write down the line number of the bug. Then write the fix.

Sources

- chisquare([50600, 49400]).pvalue is 0.000148; for 5060 vs 4940 it is 0.230. (verified: Run on SciPy 1.17.1 in this repository; matches math.erfc(sqrt(stat / 2)).)
- confint_proportions_2indep(1150, 10000, 1000, 10000) gives (0.0064, 0.0236). (verified: Run on statsmodels 0.15.0 in this repository.)
- An SRM p-value below 0.001 is a common threshold for investigation. (search-only: Fabijan et al. (2019), Diagnosing Sample Ratio Mismatch in Online Controlled Experiments, KDD.)

B9-L3 Peeking, novelty and other ways tests mislead

Concept

A test's 5% false positive rate holds only if you analyse once, at the planned sample size. Common habits break it.

Peeking. Checking results every day and stopping the first time $p < 0.05$ gives the noise ten chances to cross the line. A simulation in this course's build: two identical arms with a 10% conversion rate, 1,000 runs, 200 users

per arm per look. One look: 5.2% false positives. Five looks: 12.9%. Ten looks: 18.4%.

Fixes, in order of simplicity:

- **Fixed horizon.** Compute the sample size (lesson B9-L1), run to it, analyse once. Dashboards during the test show health metrics and the split check, not the lift.
- **Planned interim looks** with a group-sequential design, where each look uses a stricter threshold so the overall rate stays at 5%. Plan the number of looks and the boundaries before launch.
- **Always-valid methods** designed for continuous monitoring,, offered by some experimentation platforms. Use them as the platform documents them.

Novelty and learning effects. Users click a new button because it is new. The lift fades after a week. Run for at least one full weekly cycle, and compare the effect in week 1 and week 2.

Too many metrics. Twenty secondary metrics produce about one false positive per test (lesson B8-L3). Name one primary metric and a few guardrails before launch.

Stopping for a good result only. A team that stops early when B looks good but runs longer when B looks bad biases every conclusion towards B.

Interference. If treated users affect control users, through shared inventory, pricing or social features, the arms are not independent. Randomise at a level that contains the interaction, such as city or store.

Write the plan down: primary metric, MDE, sample size, duration, looks, and the decision rule. Then follow it.

Why it matters

A growth team ran 30 tests a quarter, checked each daily, and shipped any variant that reached p < 0.05. Their dashboard showed 11 winners, with a combined forecast lift of 9% in sign-ups. Sign-ups over the quarter were flat. With daily peeking over three-week tests, the false positive rate per test was far above 5%, and most of the winners were noise.

Worked example: weak code

```
1 for day in range(1, 29):
2     p = daily_p_value(day)
2     if p < 0.05:
4         ship("B")
```

```
        break
```

1. **Line 1.** Up to 28 looks at the same growing data.
2. **Line 3.** The same 5% threshold at every look: the overall false positive rate climbs well above 5%.

Worked example: strong code

```python
from dataclasses import dataclass


@dataclass(frozen=True)
class TestPlan:
    """Written before launch and stored with the results."""

    primary_metric: str
    mde_points: float
    n_per_arm: int
    min_days: int
    looks: int = 1


def ready_to_analyse(plan: TestPlan, n_per_arm_now: int, days_run: int) -> bool:
    """True only once the planned sample and duration are both reached."""
    return n_per_arm_now >= plan.n_per_arm and days_run >= plan.min_days
```

1. **Line 5.** The plan is data, fixed before launch.
2. **Line 12.** One analysis unless a sequential design was planned.
3. **Line 17.** The lift is looked at once, when the plan says so.

Plan the sample, duration and looks, analyse once at the horizon, and treat anything else as exploration.

Recap card

- **Why peeking breaks a test** Each look is another chance for noise to cross $p < 0.05$. Ten looks gave 18.4% false positives in simulation.
- **Safe alternatives to daily peeking** Fixed horizon, or planned group-sequential looks with stricter thresholds.
- **Novelty effect** Lift that fades after launch. Run full weeks and compare week 1 with later weeks.
- **Interference** Treated units affect controls. Randomise at a level that contains the interaction.

Practice

Exercise 1. Multiple choice (level 1)

You check an A/A test (no real difference) after each of 10 equal batches and stop at the first p < 0.05. In this lesson's simulation, how often did that declare a winner?

- a) About 5%
- b) About 18%
- c) About 50%
- d) Never, because the arms are identical

Why?

- r1) Each look is another chance for noise to cross the threshold.
- r2) Later batches have more users, so they are always significant.

Exercise 2. Predict the output (level 2)

A test ran 14 days. The lift in conversion was 3.1 points in week 1 and 0.4 points in week 2. What does this print?

```
week_lifts = {"week_1": 3.1, "week_2": 0.4}
fading = week_lifts["week_2"] < week_lifts["week_1"] / 2
print("novelty suspected" if fading else "stable")
```

Exercise 3. Fill the gap (level 2)

Allow analysis only when both the planned sample and the planned duration are reached.

```
return n_per_arm_now >= plan.n_per_arm _____(1) days_run >= plan._____(2)
```

Exercise 4. Write the function (level 3)

Write `peeking_false_positive_rate(rng, n_sims, n_per_look, looks, rate, z_crit=1.959964) -> float`. It simulates `n_sims` A/A tests where both arms convert at `rate`. Each arm gets `n_per_look` new users per look. At each look, run a pooled two-proportion z-test on the cumulative data. Return the share of simulations where `|z| > z_crit` at **any** look. Treat a zero standard error as z = 0. Use only the `rng` passed in.

What you need to know.

The pooled z-test. For two arms with `n` users each and `xa`, `xb` conversions: pooled rate `p = (xa + xb) / (2 * n)`, standard error `se = sqrt(p * (1 - p) * 2 / n)`, and `z = (xb / n - xa / n) / se`.

NumPy tools. These work on whole arrays, with one simulation per row:

```
# True/False draws, shape (2, 6)
hits = rng.random((2, 6)) < 0.5
# Running totals along each row
runs = hits.cumsum(axis=1)
# [1 3 5]: positions 2, 4 and 6
idx = np.arange(1, 4) * 2 - 1
# Those columns from every row
runs[:, idx]

# Divide only where y > 0, else 0: [0.5 0. ]
x, y = np.array([1.0, 2.0]), np.array([2.0, 0.0])
np.divide(x, y, out=np.zeros_like(x), where=y > 0)

# Share of rows with any True: 0.5
np.array([[True, False], [False, False]]).any(axis=1).mean()
```

`.any(axis=1)` asks "true anywhere in this row?", and `.mean()` of booleans is the share of `True`.

Your code must pass these checks:

- With one look the false positive rate must be close to 5%.
- With ten looks, count a simulation if |z| crosses the line at any look. Expect roughly 18%.
- The same seed must give the same result.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Share of A/A simulations declared significant at any of the looks.
def peeking_false_positive_rate(
    rng: np.random.Generator,
    n_sims: int,
    n_per_look: int,
    looks: int,
    rate: float,
    z_crit: float = 1.959964,
) -> float: ...
```

Starter code:

```
import numpy as np


def peeking_false_positive_rate(rng, n_sims, n_per_look, looks, rate,
    z_crit=1.959964):
    return 0.05
```

Exercise 5. Architecture design (level 4)

Stakeholders want to "see results daily" on a four-week pricing test. Choose the plan.

Options: Split check, traffic and error rates. No lift until the horizon., Daily lift with p-values.

1. What the daily dashboard shows
2. Analyses of the primary metric
3. Randomisation unit, since prices are shared in store

Sources

- A/A simulation, 1,000 runs, 200 users per arm per look, rate 10%: 5.2% false positives with 1 look, 12.9% with 5, 18.4% with 10. (verified: Simulated with NumPy 2.4.6 in this repository (seed 1).)

Track B best-practice checklist

☐ The project has a pyproject.toml, a virtual environment and pinned dependencies.

☐ ruff and mypy pass before every commit.

☐ Functions are pure where possible. Side effects sit at the edges.

☐ Dtypes are explicit. pandas code is vectorised, with no row-wise apply on large frames.

☐ Inputs are validated with pandera or pydantic at every boundary.

☐ Seeds, config and data versions are recorded for every result.

☐ Weighted estimates use the survey weights. Intervals and effect sizes go with every estimate.

☐ Preprocessing sits inside a scikit-learn Pipeline, fitted on training folds only.

☐ The metric matches the decision. Calibration and subgroup performance are checked.

☐ Every experiment is tracked in MLflow.

☐ Tests cover edge cases, including empty inputs and zeros.

☐ Tests use fixtures and parametrize, compare numbers with stated tolerances, isolate I/O, and run with coverage in CI.

☐ Every model has a model card. Every decision has a log entry.

Appendix: solutions and answer keys

Attempt each exercise before you read its solution. Track D answer keys are here too.

B9-L1 Power and sample size, before fieldwork

B9-L1 exercise 1

Hints. (1) What does power 0.25 mean? (2) How often would a real effect be missed? (3) Three times in four.

Answer. b) Very little. The test was unlikely to detect the effect even if it were real. Because: r1) Power 0.25 means a real effect of that size is missed 75% of the time.

The test could not answer the question. Report it as inconclusive and, next time, size the sample first.

Common wrong answers.

- `a`: An underpowered test misses real effects most of the time. No significance is not no effect.
- `c`: Nothing in the result points to a negative effect.
- `d`: Power is not a confidence level about the effect being zero.
- `b|r2`: Right answer, wrong reason. A well-powered non-significant result is informative.

B9-L1 exercise 2

Hints. (1) The effect sits squared in the denominator. (2) Halve the effect. (3) (1/0.5)^2.

Answer.

```
1 4
```

Half the effect, about four times the sample: 1,534 becomes roughly 6,000 per group.

Common wrong answers.

- `2`: n grows with the inverse square of the effect. Half the effect needs about four times the sample.

B9-L1 exercise 3

Hints. (1) The method that solves for whichever argument you leave out. (2) Which input is 0.8? (3) `solve_power` and `power`.

Answer.

```
h = proportion_effectsize(0.45, 0.40)
n = NormalIndPower().solve_power(effect_size=h, alpha=0.05, power=0.8)
```

`solve_power` leaves `nobs1` unset and solves for it: 1,533.05, so 1,534 per group.

Common wrong answers.

- g1 = `power`: power() returns power for a given n. solve_power finds the missing quantity, here n.

B9-L1 exercise 4

Hints. (1) Two-sided: z_a = NormalDist().inv_cdf(1 - alpha / 2). (2) Square (top / (p2 - p1)), then multiply by design_effect. (3) math.ceil last.

Answer.

```
import math
from statistics import NormalDist


def n_per_group(
    p1: float, p2: float, alpha: float = 0.05, power: float = 0.8,
    design_effect: float = 1.0
) -> int:
    """Sample per group to detect p1 vs p2 with a two-sided test.

    Args:
        p1: Baseline share.
        p2: Share under the minimum detectable effect.
        alpha: Two-sided significance level.
        power: Chance of detecting the effect if real.
        design_effect: Multiplier for weighted or clustered samples.

    Returns:
        Respondents per group, rounded up.

    Raises:
        ValueError: If p1 equals p2 or a share is outside (0, 1).
    """
    if p1 == p2 or not (0 < p1 < 1 and 0 < p2 < 1):
        raise ValueError("shares must differ and lie strictly between 0
    and 1")
    z_a = NormalDist().inv_cdf(1 - alpha / 2)
    z_b = NormalDist().inv_cdf(power)
    p_bar = (p1 + p2) / 2
```

```
    top = z_a * math.sqrt(2 * p_bar * (1 - p_bar)) + z_b * math.sqrt(p1 *
(1 - p1) + p2 * (1 - p2))
    return math.ceil((top / (p2 - p1)) ** 2 * design_effect)
```

The formula turns the four decisions into a number. The design effect scales it for the information weights remove.

B9-L1 exercise 5

Hints. (1) Two groups, 1,534 each, times 1.5. (2) Does a region need less sample to detect the same effect? (3) Decide before fieldwork.

Answer.

- Overall: about 4,600 needed with the design effect. 4,000 is close but short.
- Regions: each needs the full sample for the same MDE.
- Settle the trade-off with the client before fieldwork, in writing.

Power maths turns a budget conversation into explicit trade-offs, agreed before any data exists.

Common wrong answers.

- total = `plenty`: Two cells of 1,534, times a design effect of 1.5, need about 4,600.
- regions = `quarter`: A quarter of the sample cannot detect the same 5-point lift. Each region is its own test.
- decision = `run`: Underpowered regional reads produce noise dressed as findings.

B9-L2 Analyse an A/B test the way it was randomised

B9-L2 exercise 1

Hints. (1) Is a 1% gap on 100,000 users chance? (2) Which users went missing? (3) Find the cause first.

Answer. b) Stop and find why B lost users before trusting any result. Because: r1) A split that differs from the design means something removed users unevenly, and they may differ in outcome.

SRM means the arms no longer differ only by chance. Find the cause, fix it, and rerun.

Common wrong answers.

- `a`: With 100,000 users, a 1% gap is far beyond chance. The missing users may be the story.

- `c`: Reweighting assumes the missing users resemble the rest. They rarely do.
- `d`: Random deletion balances counts but not the kind of user lost from B.
- `b|r2`: Right answer, wrong reason. The concern is a specific cause, not a p-value rule.

B9-L2 exercise 2

Hints. (1) Same proportions, ten times the users. (2) The statistic grows with n for the same share. (3) p = 0.23, then p = 0.00015.

Answer.

```
1 True False
```

The same imbalance in share becomes decisive as the sample grows. Check SRM on counts, not on how the percentages look.

Common wrong answers.

- `True True`: The same 50.6% split is chance at 10,000 users (p = 0.23) but not at 100,000 (p = 0.00015).
- `False False`: At 10,000 users, 60 either side of 5,000 is well within chance: p = 0.23.

B9-L2 exercise 3

Hints. (1) Validity before results. (2) Units before estimates. (3) Primary before secondary.

Answer.

1. Check the split against the design
2. Aggregate to one outcome per randomised user
3. Estimate the primary metric's lift with its interval
4. Compare the interval with the planned minimum effect
5. Report secondary metrics as exploratory

Trust, then units, then the one planned answer, then everything else labelled as exploration.

Common wrong answers.

- Checking the split after seeing the lift invites excuses. Check it first.
- Estimate on per-user outcomes, so aggregate before estimating.

B9-L2 exercise 4

Hints. (1) Validate inputs first. (2) Expected counts: n * share and n * (1 - share). (3) Sum (observed - expected)^2 / expected, then erfc(sqrt(stat / 2)).

Answer.

```python
import math


def srm_p_value(n_a: int, n_b: int, expected_share_a: float = 0.5) -> float:
    """Sample ratio mismatch p-value: chi-square with 1 degree of freedom.

    Args:
        n_a: Users observed in arm A.
        n_b: Users observed in arm B.
        expected_share_a: Planned share of users in A.

    Returns:
        The p-value. Below 0.001 signals a mismatch to investigate.

    Raises:
        ValueError: On negative counts, a zero total or an invalid share.
    """
    if n_a < 0 or n_b < 0 or n_a + n_b == 0 or not 0 < expected_share_a < 1:
        raise ValueError("counts must be non-negative with a positive total, share in (0, 1)")
    n = n_a + n_b
    e_a, e_b = n * expected_share_a, n * (1 - expected_share_a)
    stat = (n_a - e_a) ** 2 / e_a + (n_b - e_b) ** 2 / e_b
    return math.erfc(math.sqrt(stat / 2))
```

The statistic compares observed with planned counts. For 1 degree of freedom the chi-square p-value equals `erfc(sqrt(stat / 2))`, so no SciPy is needed.

B9-L2 exercise 5

Hints. (1) Which users does line 6 drop? (2) Is the same rule applied to A? (3) Keep everyone as assigned.

Answer. The bug is on line 6.

Fixed version:

```python
import pandas as pd


def conversion_by_arm(users: pd.DataFrame) -> dict[str, float]:
    """Conversion rate per assigned arm. One row per randomised user."""
    return users.groupby("arm")["converted"].mean().to_dict()
```

Filtering B on `saw_new_page` keeps its most engaged users and drops the rest, while A keeps everyone. The arms are no longer comparable. Analyse everyone as assigned.

Common wrong answers.

- Line 7: Grouping by arm is right. The rows reaching it are the problem.

B9-L3 Peeking, novelty and other ways tests mislead

B9-L3 exercise 1

Hints. (1) How many chances does noise get? (2) One look: 5%. Ten looks: more. (3) The simulation gave 18.4%.

Answer. b) About 18% Because: r1) Each look is another chance for noise to cross the threshold.

Repeated looks multiply the chances of a false positive. Fix the horizon or use a planned sequential design.

Common wrong answers.

- `a`: 5% holds for one planned look. Ten looks gave 18.4% in the simulation.
- `c`: Too high for 10 looks. Unlimited looking pushes it higher still.
- `d`: Identical arms still produce p < 0.05 by chance. That is what a 5% level means.
- `b|r2`: Right number, wrong reason. More users do not create significance when there is no effect.

B9-L3 exercise 2

Hints. (1) Half of 3.1 is 1.55. (2) Is 0.4 below that? (3) Yes.

Answer.

```
1 novelty suspected
```

A lift that fades fast is often novelty. Report the later weeks, or run longer before deciding.

Common wrong answers.

- `stable`: 0.4 is less than half of 3.1, so the effect is fading.

B9-L3 exercise 3

Hints. (1) Both, not either. (2) The plan field for duration. (3) `and` and `min_days`.

Answer.

```python
return n_per_arm_now >= plan.n_per_arm and days_run >= plan.min_days
```

Sample size gives power. Duration covers weekly cycles and novelty. The analysis needs both.

Common wrong answers.

- g1 = `or`: or lets a big day-2 sample through before a full weekly cycle. Both conditions must hold.

B9-L3 exercise 4

Hints. (1) Draw conversions for all users at once, then take cumulative sums along each row. (2) Pick the columns at the end of each look: `np.arange(1, looks + 1) * n_per_look - 1`. (3) A simulation counts if `(np.abs(z) > z_crit).any(axis=1)`.

Answer.

```python
import numpy as np


def peeking_false_positive_rate(
    rng: np.random.Generator,
    n_sims: int,
    n_per_look: int,
    looks: int,
    rate: float,
    z_crit: float = 1.959964,
) -> float:
    """Share of A/A simulations declared significant at any of the looks.

    Args:
        rng: Seeded generator.
        n_sims: Number of simulated tests.
        n_per_look: New users per arm between looks.
        looks: Number of analyses.
        rate: True conversion rate in both arms.
        z_crit: Two-sided threshold for each look.

    Returns:
        The false positive rate across simulations.
    """
    total = looks * n_per_look
    a = (rng.random((n_sims, total)) < rate).cumsum(axis=1)
    b = (rng.random((n_sims, total)) < rate).cumsum(axis=1)
    idx = np.arange(1, looks + 1) * n_per_look - 1
    n = idx + 1
    xa, xb = a[:, idx], b[:, idx]
    pooled = (xa + xb) / (2 * n)
    se = np.sqrt(pooled * (1 - pooled) * 2 / n)
    diff = (xb - xa) / n
    z = np.divide(diff, se, out=np.zeros_like(diff), where=se > 0)
```

```
    return float((np.abs(z) > z_crit).any(axis=1).mean())
```

Cumulative sums give every look's data from one draw. Counting a simulation when any look crosses the threshold shows how peeking inflates false positives.

B9-L3 exercise 5

Hints. (1) What should people see before the answer is ready? (2) How many looks, at what threshold? (3) Who shares the treatment?

Answer.

- Daily views of health, not of the answer.
- One planned analysis, or a group-sequential design agreed before launch.
- Randomise by store, because visitors in the same store share prices.

Separate monitoring from deciding, plan the looks, and randomise at the level where treatment is shared.

Common wrong answers.

- dashboard = `lift`: A daily lift invites stopping on noise. Show health metrics until the horizon.
- looks = `daily`: 28 looks at 5% each inflate the false positive rate far above 5%.
- unit = `visitor`: Visitors in one store see the same shelf price. Randomising visitors mixes arms within a store.